

Pima Indians Diabetes Database

By Elizabeth Hui Shen

1- Load the dataset, generate a numeric statistical summary of all its variables.

```
install.packages("readr")
library(readr)
library(dplyr)
                                                    # download data from a website

url <- "https://drive.google.com/uc?id=1qDkGULtjrXw-H4OPXgPZH6PnVcPBHzYK"
file_path <- "pima_indians_doabetes.csv"

download.file(url,destfile = file_path, mode = "wb")

                                                                # load csv to a df
diabetes_data = read.csv(file_path)

                                                            # display first rows of the df
head(diabetes_data)

                                                    # generate a numeric statistical summary

summary(diabetes_data)
```

2- The dataset contains missing values. Sometimes those values are blank and some other times a question mark has been used to indicate them. If any variable has more than 20% of its values missing, drop that variable.

```
#  check missing values
missing_values <- sapply(diabetes_data, function(x) sum(is.na(x) | x == "" | x == "?"))

# set the threshold value
drop_value <- 0.2* nrow(diabetes_data)

# Drop variables with more than 20% missing values

keep = missing_values[missing_values <= drop_value]

# retrieve keep variables names
variables_to_keep <- names(keep)
# save to a df
diabetes_data <- diabetes_data[, variables_to_keep]

# review the filtered df:
summary(diabetes_data)
nrow(diabetes_data)
ncol(diabetes_data)                    # successfully dropped columns
```

3- For the remaining variables, Replace all missing values in each variable by the minimum value in that variable.
Now the dataset should be full without any missing values.

```
install.packages("tidyr")

library(tidyr)

                                                    # Replace all other missing value

                                                                # as "NA"


diabetes_data[diabetes_data == "?"]<- NA

diabetes_data[diabetes_data == ""] <- NA

                                                                    # Replace all na with

                                                    # minimum value of that variable


for(i in 1:ncol(diabetes_data)){

    diabetes_data[is.na(diabetes_data[,i]), i] <- min(diabetes_data[,i], na.rm = TRUE)

 }

                                                # check and convert all no numeric columns to numeric


diabetes_data$Glucose=as.numeric(diabetes_data$Glucose)

class(diabetes_data$Glucose) # numeric


diabetes_data$DiabetesPedigreeFunction=as.numeric(diabetes_data$DiabetesPedigr
eeFunction)

class(diabetes_data$DiabetesPedigreeFunction) # numeric


diabetes_data$Age=as.numeric(diabetes_data$Age)

class(diabetes_data$Age) # numeric
```

4- Some variables will contain 0 values but those values don't make sense. In this dataset, 0 values only make sense in the Pregnancies and Outcome columns. Drop any row that contains one or more 0's in any column(s) other than Pregnancies and Outcome.

```
                                                    # columns to keep
columnkeep=c( "Pregnancies" ,"Outcome" )

                                                        # chain functions
diabetes_data <-diabetes_data %>%
filter_at(vars(-Pregnancies, -Outcome), all_vars(. != 0))
```

5- Group the data based on the Outcome column and generate the mean and median values for all other columns.

```
grouped_df <- aggregate(. ~ Outcome, data = diabetes_data, FUN = function(x)
c(mean = mean(x), median = median(x)))

# another solution/chain funtion
group_df2=diabetes_data%>%
 group_by(Outcome) %>%
 summarise_all(list(mean = mean, median = median))
```

6- Generate a heatmap plot that shows the correlation values between all variables in the dataset.

```
library(dplyr)
library(ggplot2)
library(reshape2)
library(corrplot)

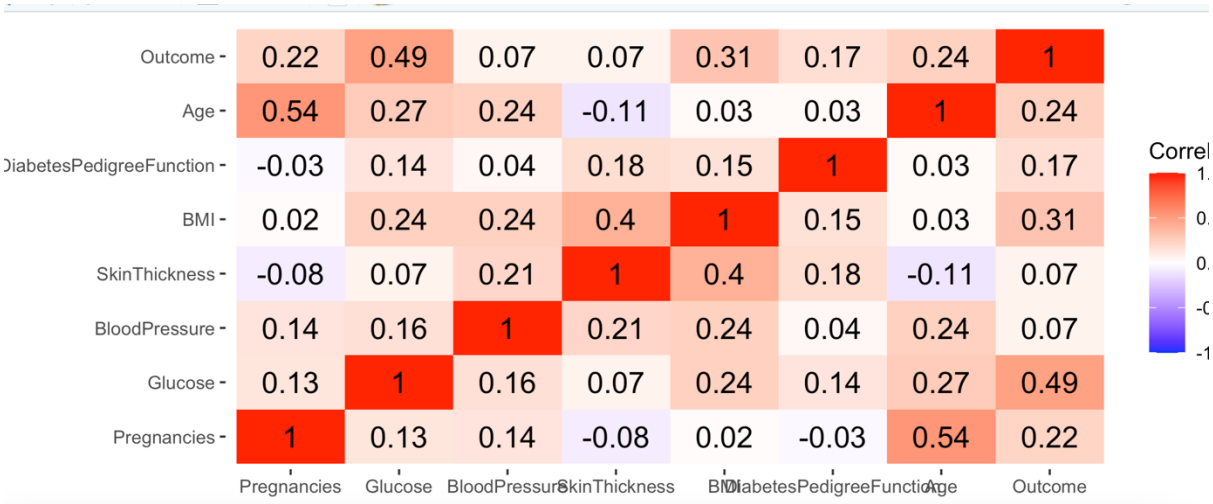
                                    # generate a correlation matrix
```

```
correlation_matrix <- cor(diabetes_data)

                                        #calculate correlation coefficients
                                        #rounded to 2 decimal places
cor_df <- round(cor(diabetes_data), 2)

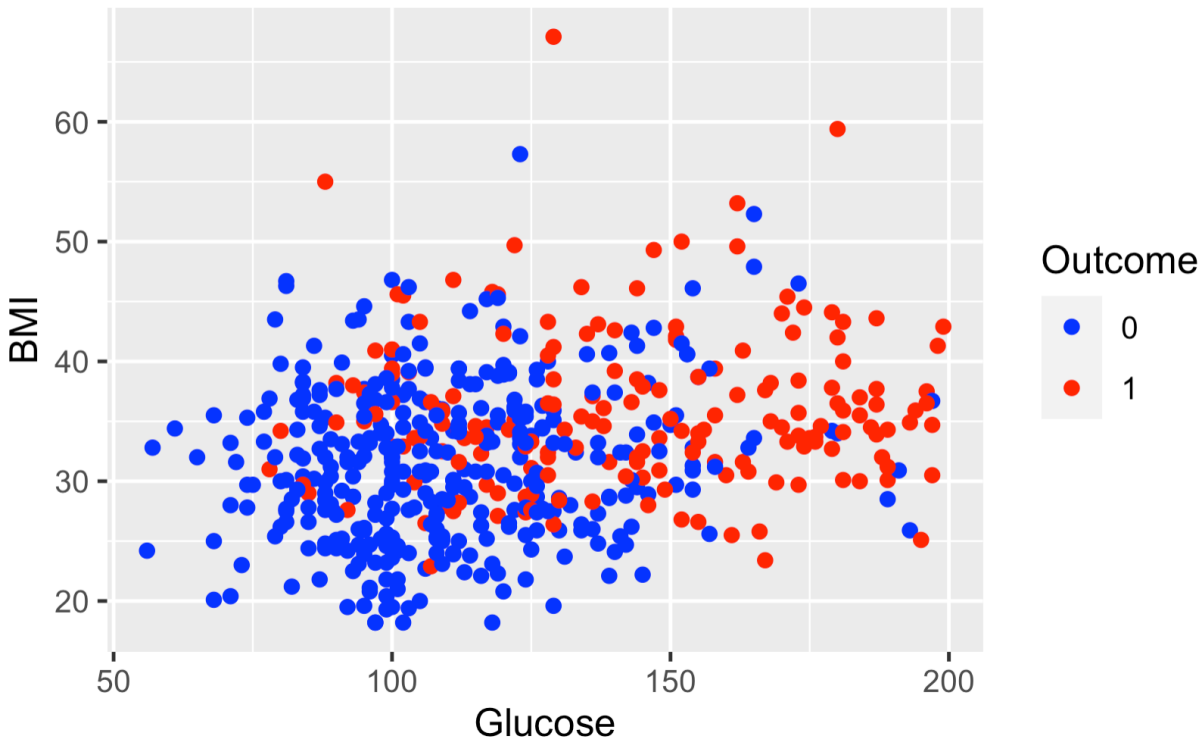
                                                #melt the data frame
melted_cor <- melt(cor_df)

                                                        # generate plot
ggplot(data = melted_cor, aes(x=Var1, y=Var2, fill=value)) +
  geom_tile() +
  geom_text(aes(Var2, Var1, label = value), size = 5) +
  scale_fill_gradient2(low = "blue", high = "red",
                       limit = c(-1,1), name="Correlation") +
  theme(axis.title.x = element_blank(),
        axis.title.y = element_blank(),
        panel.background = element_blank())
```



7- Generate a scatter plot showing Glucose in the x-axis and BMI in the y-axis. Colourise the markers by Outcome (e.g. markers representing diabetic people should be red and markers representing non-diabetic people should be blue).

```
# generate scatter pot
ggplot(diabetes_data, aes(x = Glucose, y = BMI, color = as.factor(Outcome))) +
  geom_point() +
  labs(x = "Glucose", y = "BMI", color = "Outcome") +
  scale_color_manual(values = c("blue", "red"))
```



8- Create a new column in the dataset to categorize BMI into groups.

```
# BMI category
diabetes_data$BMI_Category <- cut(diabetes_data$BMI, breaks = c(-Inf, 23, 29, Inf), labels = c("Underweight", "Normal Weight", "Overweight"))
```

	BMI	DiabetesPedigreeFunction	Age	Outcome	BMI_Category
35	33.6	0.627	50	1	Overweight
29	26.6	0.351	31	0	Normal Weight
23	28.1	0.167	21	0	Normal Weight
35	43.1	2.288	33	1	Overweight
32	31.0	0.248	26	1	Overweight
45	30.5	0.158	53	1	Overweight
23	30.1	0.398	59	1	Overweight
19	25.8	0.587	51	1	Normal Weight
47	45.8	0.551	31	1	Overweight
38	43.3	0.183	33	0	Overweight
30	34.6	0.529	32	1	Overweight

9- Group the data by BMI category and calculate the percentage of individuals with diabetes within each category.

```
# group BMI
BMI_percentage_by_category <- aggregate(Outcome ~ BMI_Category, data = diabetes_data, function(x) mean(x == 1) * 100)

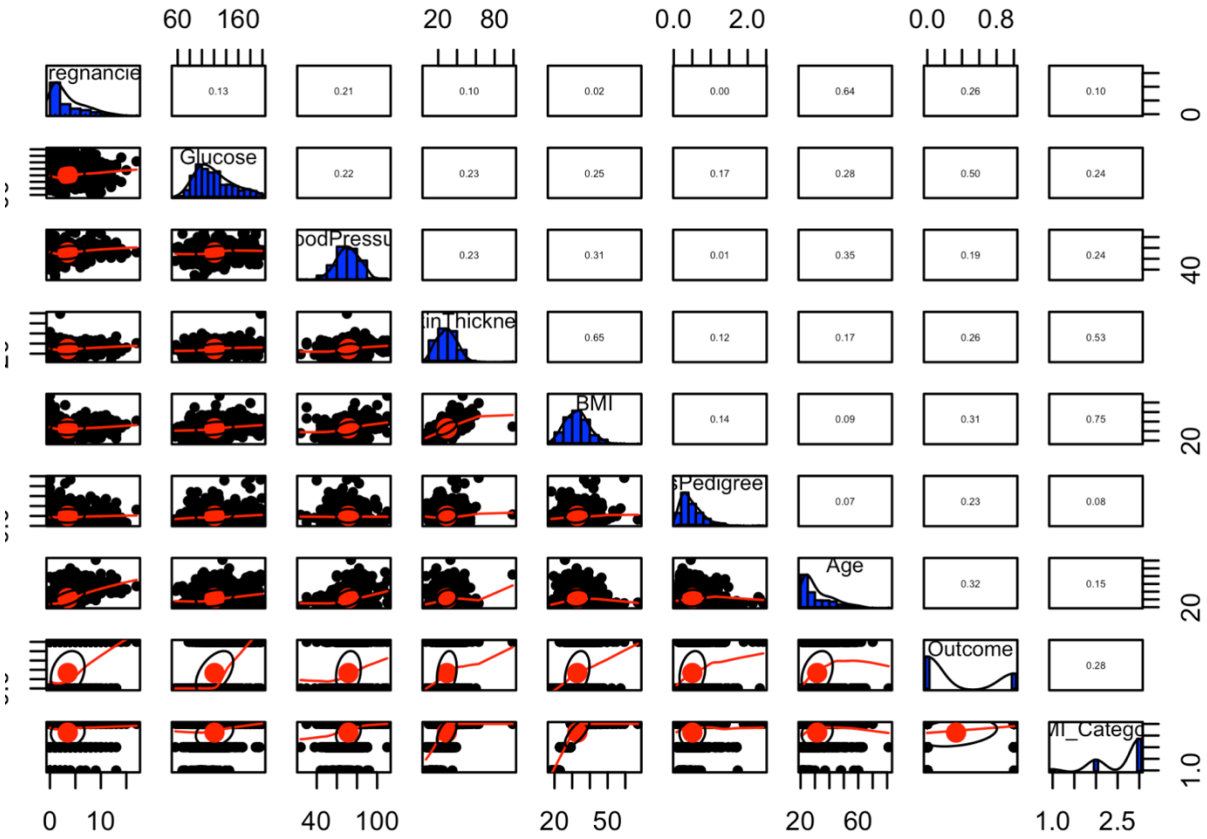
colnames(BMI_percentage_by_category) <- c('BMI_Category','Percentage')
```

	BMI_Category	Percentage
1	Underweight	3.12500
2	Normal Weight	16.05839
3	Overweight	42.16216

10- Generate a scatter matrix plot showing all variables vs each other with histograms on the diagonal.

```
library(ellipse)
library(ggplot2)
library(psych)
                                                    # generate plot

pairs.panels(diabetes_data,
        method = "pearson", # correlation method
        hist.col = "blue",
        density = TRUE,  # show density plots
        ellipses = TRUE # show correlation ellipses
)
```




thank you
danke
謝謝
gracias
merci
obrigado
dziękuję
bedankt
teşekkür ederim
go raibh maith agat
sukriya
kop khun krap
terima kasih
arigatō
dakujem
grazie
tapadh leat
mochchakkeram
ngiyabonga
спасибо
рахмат
Баярлалаа
dank je
hvala
mauruuru
sagolun
takk
tack
merci
ευχαριστώ
감사합니다
ধন্যবাদ
ありがとう
мерси
danke
thank you
gracias
merci
obrigado
dziękuję
bedankt
teşekkür ederim
go raibh maith agat
sukriya
kop khun krap
terima kasih
arigatō
dakujem
grazie
tapadh leat
mochchakkeram
ngiyabonga
спасибо
рахмат
Баярлалаа
dank je
hvala
mauruuru
sagolun
takk
tack
merci
ευχαριστώ
감사합니다
ধন্যবাদ
ありがとう
мерси